WEEK-8 KAFKA LAB EXPERIMENT

Simulate Fault Tolerance by Shutting Down One Broker

1. Aim

To simulate fault tolerance by shutting down one Kafka broker and observing the behavior of a replicated Kafka cluster.

2. Requirements

- Ubuntu Linux
- Docker
- Docker Compose
- Apache Kafka (confluentinc/cp-kafka:7.6.1 image)
- Apache ZooKeeper (confluentinc/cp-zookeeper:7.6.1 image)
- Ubuntu Terminal

3. Directory Structure

```
week-8/
│
├── docker-compose.yml
├── create-topic.sh
├── produce.sh
├── consume.sh
└── README.md
```

4. Create the Week-8 Directory

```
cd ~/Desktop
mkdir -p week-8
cd week-8
pwd
```

Expected:

```
/home/<username>/Desktop/week-8
```

5. Create Required Files

```
touch docker-compose.yml create-topic.sh produce.sh consume.sh README.md
tree
```

If tree is not installed:

```
sudo apt update
sudo apt install tree -y
tree
```

6. docker-compose.yml

Open the file:

```
nano docker-compose.yml
```

Paste the following:

```
services:

  zookeeper:
    image: confluentinc/cp-zookeeper:7.6.1
    container_name: zookeeper
    environment:
      ZOOKEEPER_CLIENT_PORT: 2181
    ports:
      - "2181:2181"

  kafka-1:
    image: confluentinc/cp-kafka:7.6.1
    container_name: kafka-1
    depends_on:
      - zookeeper
    ports:
      - "9092:9092"
    environment:
      KAFKA_BROKER_ID: 1
      KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka-1:9092
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 2
      KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR: 2
      KAFKA_TRANSACTION_STATE_LOG_MIN_ISR: 1

  kafka-2:
    image: confluentinc/cp-kafka:7.6.1
    container_name: kafka-2
    depends_on:
      - zookeeper
    ports:
      - "9093:9092"
    environment:
      KAFKA_BROKER_ID: 2
      KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka-2:9092
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 2
      KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR: 2
      KAFKA_TRANSACTION_STATE_LOG_MIN_ISR: 1
```

Save: Ctrl+O → Enter. Exit: Ctrl+X.

7. Start the Kafka Cluster

```
cd ~/Desktop/week-8
docker compose up -d
```

If your Ubuntu installation uses the older Compose command, use:

```
docker-compose up -d
```

Expected output:

```
[+] Running 4/4
 ✔ Network week-8_default  Created
 ✔ Container zookeeper  Started
 ✔ Container kafka-1    Started
 ✔ Container kafka-2    Started
```

8. Check Running Brokers

```
docker ps --format "table {{.Names}}\t{{.Status}}"
```

Expected:

```
NAMES       STATUS
kafka-1     Up ...
kafka-2     Up ...
zookeeper   Up ...
```

9. Check Kafka Logs

```
docker logs kafka-1 --tail 20
docker logs kafka-2 --tail 20
```

10. Create create-topic.sh

```
nano create-topic.sh
```

Paste:

```
#!/bin/bash

echo "Creating Kafka topic..."

docker exec kafka-1 kafka-topics \
  --create \
  --topic test-topic \
  --bootstrap-server kafka-1:9092 \
  --partitions 1 \
  --replication-factor 2

echo "Topic creation completed."
```

Save and exit, then make it executable:

```
chmod +x create-topic.sh
ls -l create-topic.sh
```

11. Create the Replicated Topic

```
./create-topic.sh
```

Expected:

```
Creating Kafka topic...
Created topic test-topic.
Topic creation completed.
```

If you see 'Topic test-topic already exists', the topic has already been created and you can continue.

12. Verify the Topic

```
docker exec kafka-1 kafka-topics \
  --list \
  --bootstrap-server kafka-1:9092
```

Expected:

```
test-topic
```

13. Describe the Topic Before Failure

```
docker exec kafka-1 kafka-topics \
  --describe \
  --topic test-topic \
  --bootstrap-server kafka-1:9092
```

Expected approximately:

```
Topic: test-topic
PartitionCount: 1
ReplicationFactor: 2
Partition: 0
Leader: 1
Replicas: 1,2
Isr: 1,2
```

Observation: Broker 1 (usually; it may be broker 2) is the leader, both brokers contain replicas, and both brokers are in the ISR (In-Sync Replicas).

14. Create produce.sh

```
nano produce.sh
```

Paste:

```
#!/bin/bash

echo "Producing messages to test-topic..."
```

```
docker exec -i kafka-1 kafka-console-producer \
  --topic test-topic \
  --bootstrap-server kafka-1:9092 <<EOF
Message-1
Message-2
Message-3
Message-4
Message-5
EOF

echo "Messages produced successfully."
```

Make it executable:

```
chmod +x produce.sh
```

15. Produce Messages

```
./produce.sh
```

Expected:

```
Producing messages to test-topic...
Messages produced successfully.
```

16. Create consume.sh

```
nano consume.sh
```

Paste:

```
#!/bin/bash

echo "Consuming messages from test-topic..."

docker exec kafka-2 kafka-console-consumer \
  --topic test-topic \
  --bootstrap-server kafka-2:9092 \
  --from-beginning \
  --timeout-ms 10000
```

Make it executable:

```
chmod +x consume.sh
```

17. Consume Messages

```
./consume.sh
```

Expected:

```
Message-1
Message-2
```

```
Message-3
Message-4
Message-5
```

18. Simulate Broker Failure

Stop Kafka Broker 1:

```
docker stop kafka-1
```

Expected:

```
kafka-1
```

Check the remaining running containers:

```
docker ps --format "table {{.Names}}\t{{.Status}}"
```

Expected:

```
NAMES       STATUS
kafka-2     Up ...
zookeeper   Up ...
```

19. Observe Cluster Behavior After Broker Failure

Important: kafka-1 is stopped, so commands must now be executed through kafka-2.

```
docker exec kafka-2 kafka-topics \
  --describe \
  --topic test-topic \
  --bootstrap-server kafka-2:9092
```

Expected approximately:

```
Topic: test-topic
PartitionCount: 1
ReplicationFactor: 2
Partition: 0
Leader: 2
Replicas: 1,2
Isr: 2
```

Observation: Broker 2 automatically becomes the leader. The topic remains available through the surviving broker.

20. Produce Messages After Broker-1 Failure

```
docker exec -it kafka-2 kafka-console-producer \
  --topic test-topic \
  --bootstrap-server kafka-2:9092
```

Enter:

```
Fault-Test-1
Fault-Test-2
Fault-Test-3
```

Press Ctrl+C to exit the producer.

21. Consume Messages After Broker-1 Failure

```
docker exec kafka-2 kafka-console-consumer \
  --topic test-topic \
  --bootstrap-server kafka-2:9092 \
  --from-beginning \
  --timeout-ms 10000
```

Expected approximately:

```
Message-1
Message-2
Message-3
Message-4
Message-5
Fault-Test-1
Fault-Test-2
Fault-Test-3
```

22. Restart Broker-1

```
docker start kafka-1
sleep 30
```

Check the containers:

```
docker ps --format "table {{.Names}}\t{{.Status}}"
```

Expected:

```
NAMES       STATUS
kafka-1     Up ...
kafka-2     Up ...
zookeeper   Up ...
```

23. Check ISR After Recovery

```
docker exec kafka-1 kafka-topics \
  --describe \
  --topic test-topic \
  --bootstrap-server kafka-1:9092
```

After broker 1 has synchronized, expected approximately:

```
Topic: test-topic
PartitionCount: 1
ReplicationFactor: 2
```

```
Partition: 0
Leader: 2
Replicas: 1,2
Isr: 1,2
```

The leader may remain broker 2. The important recovery condition is ISR: 1,2, which confirms that broker 1 has rejoined the synchronized replicas.

24. Complete Execution Commands

```
cd ~/Desktop
mkdir -p week-8
cd week-8

touch docker-compose.yml create-topic.sh produce.sh consume.sh README.md

nano docker-compose.yml

docker compose up -d

docker ps --format "table {{.Names}}\t{{.Status}}"

chmod +x create-topic.sh produce.sh consume.sh

./create-topic.sh

docker exec kafka-1 kafka-topics \
  --describe \
  --topic test-topic \
  --bootstrap-server kafka-1:9092

./produce.sh
./consume.sh

docker stop kafka-1

docker exec kafka-2 kafka-topics \
  --describe \
  --topic test-topic \
  --bootstrap-server kafka-2:9092

docker start kafka-1
sleep 30

docker exec kafka-1 kafka-topics \
  --describe \
  --topic test-topic \
  --bootstrap-server kafka-1:9092
```

25. Final Directory Structure

```
week-8/
|
```

```
├── docker-compose.yml
├── create-topic.sh
├── produce.sh
├── consume.sh
└── README.md

tree
```

26. Observation Table

Stage	Leader	Replicas	ISR
Before failure	1	1,2	1,2
Broker-1 stopped	2	1,2	2
Broker-1 restarted and synchronized	2*	1,2	1,2

* The leader can remain broker 2 after broker 1 returns. The important condition is that ISR becomes 1,2 again.

27. Important Notes

- Use 'docker compose' on modern Docker installations. If unavailable, use 'docker-compose'.
- Do not run docker exec on kafka-1 while kafka-1 is stopped.
- After kafka-1 fails, use kafka-2 to describe the topic and verify the new leader.
- Replication factor 2 means the partition has two replicas.
- ISR means In-Sync Replicas.
- After recovery, ISR should return to 1,2 after synchronization.
- The leader does not have to return to broker 1 for the experiment to be successful.

28. Result

The fault tolerance of a two-broker Kafka cluster was successfully simulated. When kafka-1 was stopped, Kafka automatically elected kafka-2 as the leader for the replicated partition. The topic remained available through the surviving broker. After kafka-1 was restarted, it rejoined the cluster and, after synchronization, returned to the ISR. Thus, Kafka broker fault tolerance and replica recovery were successfully demonstrated.

29. Lab Record Conclusion

Hence, the Kafka cluster successfully tolerated the failure of one broker using replicated partitions. The experiment demonstrated leader election, ISR behavior, continued message availability, and replica recovery after the failed broker was restarted.